

Introduction to Reverse Engineering

or: how to make everything open source

razm0

EPFL – polygl0ts

Septembre 18
2026



What's rev?

- ▶ Stands for reverse engineering
- ▶ You have a binary, no source code and you want to do understand what it does
- ▶ For example:
 - ▶ Bypassing license checks, DRM, etc.
 - ▶ Vulnerability research
 - ▶ Malware analysis
 - ▶ ... and CTFs!

What are rev techniques?

- ▶ Static analysis (looking at the binary without running it)
- ▶ Dynamic analysis (running the binary and observing its behavior, uses a debugger and things like code patching)
- ▶ Best: both! (if possible)

What's a binary?

Binary files are the compiled version of source code

More precisely:

```
> file /bin/ls
/bin/ls: ELF 64-bit LSB pie executable, x86-64,
version 1 (SYSV), dynamically linked, interpreter
/lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=baba64ae8c7edfd5d7c2b72b425d45ab72a46785,
for GNU/Linux 4.4.0, stripped
```

```
> hexdump -C /bin/ls | head -n 5
```

00000000	7f 45 4c 46 02 01 01 00	00 00 00 00 00 00 00 00	. ELF
00000010	03 00 3e 00 01 00 00 00	30 51 00 00 00 00 00 00	.. >.....0Q.....
00000020	40 00 00 00 00 00 00 00	c0 13 02 00 00 00 00 00	@
00000030	00 00 00 00 40 00 38 00	0d 00 40 00 1c 00 1b 00	@.8...@.....
00000040	06 00 00 00 04 00 00 00	40 00 00 00 00 00 00 00	@



What's a binary? (bad software version)

```
$ file notepad.exe
```

```
notepad.exe: PE32+ executable (GUI) x86-64, for MS Windows,  
6 sections
```

```
$ hexdump -C notepad.exe | head -n 5
```

```
00000000  4d 5a 90 00 03 00 00 00  04 00 00 00 ff ff 00 00  MZ.....  
00000010  b8 00 00 00 00 00 00 00  40 00 00 00 00 00 00 00  .....@.....  
00000020  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  .....  
00000030  00 00 00 00 00 00 00 00  00 00 00 00 f8 00 00 00  .....  
00000040  0e 1f ba 0e 00 b4 09 cd  21 b8 01 4c cd 21 54 68  .....!.L!Th
```

Which one to focus ?

Value your sanity ?

▶ Unix (ELF)

Don't value your sanity ?

▶ Windows (MZ)

So which one will we chose today?

▶ Windows (MZ)

MsoTriState Enum

Definition

Namespace: [Microsoft.Office.Core](#)
Assembly: [office.dll](#)

Specifies a tri-state Boolean value.

C++ ▾

[Copy](#)

```
public enum class MsoTriState
```

Inheritance [Enum](#) → [MsoTriState](#)

Fields

[Expand table](#)

Name	Value	Description
msoTriStateToggle	-3	Not supported.
msoTriStateMixed	-2	Not supported.
msoTrue	-1	True.
msoFalse	0	False.
msoCTrue	1	Not supported.

Applies to

Product	Versions
Office primary interop assembly	Latest

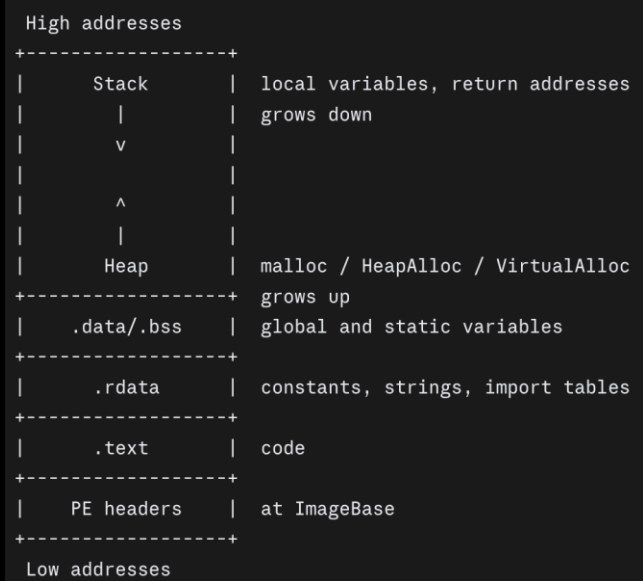


What's PE ?

PE file format

- ▶ Dos header / PE format
- ▶ Import / export table
- ▶ Section table
- ▶ Sections (map segments to sections)
- ▶ Dynamic linking

What's a binary made of ?



Understand a binary

What is a register ?

- Hold value and pointer

Register	Usage in x64 Windows
RAX	Return value. Holds the syscall number in ntdll stubs. Volatile.
RCX, RDX, R8, R9	Arguments 1-4 (5+ on the stack after 32 bytes of shadow space). RCX holds <code>this</code> in C++ methods. Volatile.
R10, R11	Scratch. In syscalls, R10 takes RCX's value and R11 receives saved RFLAGS. Volatile.
RBX, RBP, RSI, RDI, R12-R15	General purpose, preserved across calls. RBP may serve as frame pointer, RSI/RDI as source/destination in string ops. Non-volatile.
RSP	Stack pointer, 16-byte aligned before a <code>call</code> . Non-volatile.
RIP	Instruction pointer, used for RIP-relative access to globals and strings.
RFLAGS	Status flags for conditional jumps. TF is used for single-stepping.
GS	User mode: TEB (<code>gs:[0x30]</code>), PEB (<code>gs:[0x60]</code>). Kernel mode: KPCR.
XMM0-XMM3	Float arguments 1-4. XMM0 holds float return values. Volatile.
XMM4-XMM5	Scratch SSE. Volatile.
XMM6-XMM15	SSE, often used for inlined memcpy/memset. Non-volatile.



Understand a binary

What is an instruction ?

► The backbones of a program

```
int add(int a, int b)
{
    return a + b;
}
```

```
mov     DWORD PTR [rsp+10h], edx
mov     DWORD PTR [rsp+8h], ecx
mov     eax, DWORD PTR [rsp+10h]
mov     ecx, DWORD PTR [rsp+8h]
add     ecx, eax
mov     eax, ecx
ret
```

Instruction	Usage
MOV	Copies data between registers, memory, and constants (<code>mov rax, [rbx+8]</code>). The most common instruction by far.
LEA	Loads an address without dereferencing it. Used for pointers (<code>lea rcx, [rip+str]</code>) and for quick arithmetic (<code>lea rax, [rcx+rcx*2]</code>).
PUSH / POP	Save and restore registers on the stack, typically in function prologues and epilogues.
CALL / RET	CALL pushes the return address and jumps to a function. RET pops that address and returns.
CMP / TEST	Set flags without storing a result. CMP subtracts and TEST ANDs. <code>test rax, rax</code> is the classic check for zero or NULL.
JMP / Jcc	Unconditional jump, or a conditional one (<code>je</code> , <code>jne</code> , <code>jg</code> , <code>jle</code> ...) that follows CMP or TEST. These form the <code>if</code> and loop structures.
ADD / SUB	Arithmetic. <code>sub rsp, 0x28</code> in a prologue allocates stack space, including the shadow space.
XOR	<code>xor eax, eax</code> is the idiomatic way to zero a register. XOR in a loop over a buffer often points to simple encryption or obfuscation.
AND / OR / SHL / SHR	Bit manipulation used for masks, flags, alignment, and fast multiplication or division by powers of 2. Common in hashing and crypto routines.



Tools

- ▶ `file` to figure out what kind of binary it is
- ▶ `strings` to get printable strings from the binary
- ▶ `ltrace/strace` to trace library/syscalls
- ▶ `gdb` for dynamic analysis (by the grace of the lord use one of the mentioned bellow)
 - ▶ GEF
 - ▶ pwndbg
 - ▶ PEDA
- ▶ `objdump/readelf` for static analysis
- ▶ IDA Pro, Ghidra, radare2, Binary Ninja for disassembly/decompilation
- ▶ `pwntools` for exploit development

decompilation

go from disassembly to "source code"

```
void processEntry entry(undefined8 param_1,undefined8 param_2){  
    undefined auStack_8 [8];  
  
    __libc_start_main(FUN_20,param_2,&stack8,0,0,param_1,auStack_8);  
    do {  
        /* WARNING: Do nothing block with infinite loop */  
    } while( true );  
}
```



Ghidra / Ida / Binary Ninja

- ▶ <https://ghidra-sre.org/>
- ▶ Developed by the NSA.open source, free
- ▶ There are other tools like
 - ▶ [IDA \(Free\)](#)
 - ▶ [Binary Ninja](#)
 - ▶ [radare2](#)
 - ▶ [Cutter \(GUI for radare2\)](#)

▶ Installation and demo !

Extra: linux (ELF 64) convention

Register	Usage in x64 ELF (System V AMD64 ABI)
RAX	Return value (RDX:RAX for 128-bit). Syscall number. For variadic calls, AL holds the number of XMM registers used. Volatile.
RDI, RSI, RDX, RCX, R8, R9	Arguments 1-6. Arguments 7+ go on the stack. Volatile.
R10, R11	Scratch. In syscalls, R10 replaces RCX as the 4th argument, and <code>syscall</code> overwrites RCX and R11. Volatile.
RBX, RBP, R12-R15	General purpose, preserved across calls. RBP may serve as frame pointer. Non-volatile.
RSP	Stack pointer, 16-byte aligned before a <code>call</code> . No shadow space, but leaf functions can use a 128-byte red zone below RSP. Non-volatile.
FS	Thread-local storage. <code>fs:[0x28]</code> holds the stack canary.
XMM0-XMM7	Float arguments 1-8. XMM0 (and XMM1) hold float return values. Volatile.
XMM8-XMM15	Scratch SSE. Volatile (unlike Windows, no XMM register is preserved).



Extra: Windows you might wanna look into

- ▶ DOS
- ▶ TLS callback
- ▶ Registry keys